Universidad de Guadalajara

Centro Universitario de Ciencias Exactas e Ingenierías

Ingeniería informática



Algoritmos metaheurísticos

Reporte de Práctica: Algoritmo Genético Binario

Título: Actividad 2: Implementación de Algoritmo Genético Binario

Tema: Optimización de Funciones Matemáticas (1D y Ackley)

Ángel Jean Fernández Argoul

1. Marco Teórico

El Algoritmo Genético es una técnica de optimización basada en los principios de la evolución natural. En este enfoque, una **solución candidata** al problema se denomina **individuo** o **cromosoma**.

1.1 Codificación y Población Inicial

Para resolver problemas matemáticos, se define una **codificación binaria**, donde las variables reales se representan mediante cadenas de bits (0s y 1s). El algoritmo comienza generando una **solución inicial** de forma aleatoria, cuyo tamaño depende del problema y no debe cubrir necesariamente todas las combinaciones posibles.

1.2 Evaluación (Fitness)

Cada individuo debe ser decodificado para evaluar su aptitud o **fitness**. En problemas de minimización, buscamos que el valor de la función objetivo sea el menor posible. El fitness guía la evolución, permitiendo distinguir qué soluciones son mejores que otras.

1.3 Selección

El proceso de selección determina qué individuos transmitirán sus genes. Existen varios métodos:

- **Método de la Ruleta:** Se basa en la probabilidad proporcional al fitness ($\sum p$), donde se selecciona un individuo si un número aleatorio cae en su rango acumulado.
- **Selección por Torneo:** Se define un subgrupo de X individuos aleatorios y se selecciona al mejor de ellos. Este es el método implementado en esta práctica por su eficiencia.

1.4 Cruza (Crossover)

La cruza combina la información de dos padres. Se selecciona un **Punto de Cruce** (Vc) aleatorio dentro del cromosoma.

- **Padre 1:** `100110`
- **Padre 2:** `001101`

Si Vc corta después del segundo bit, los hijos resultantes intercambian las colas de los padres, generando nuevas combinaciones como `001110` y `100101`.

1.5 Mutación

Para mantener la diversidad, se aplica un **Porcentaje de Mutación (PM)**. Se recorre cada bit del individuo y se genera un valor aleatorio Mr. Si Mr<PM, el bit se altera. La operación matemática utilizada para invertir el bit es:

$$Indiv[i] = |Indiv[i] - 1|$$

Esto convierte un 0 en 1 y viceversa.

1.6 Elitismo y Condición de Paro

El algoritmo incluye **elitismo**, conservando al mejor individuo de cada generación. El proceso se detiene al cumplir una condición de paro, como alcanzar un número máximo de iteraciones.

2. Explicación del Código Implementado

El programa fue desarrollado en **C++**.

2.1 Estructura del Individuo

Se utilizó un `struct` que contiene un vector de enteros para los genes (genotipo) y un vector de doubles para las variables decodificadas (fenotipo).

```cpp
struct Individuo {
    std::vector<int> genes; // Genotipo binario
    std::vector<double> variables; // Fenotipo real
    double fitness; // Valor de la función
};
```

2.2 Decodificación (Binario a Real)

Se implementó una función de mapeo lineal. Destaca el uso de desplazamiento de bits (<<) para convertir el arreglo binario a decimal eficientemente.

```cpp
// Decodificacion: Convierte binario a real
Windsurf: Refactor | Explain | ×
double decodificar(const std::vector<int>& genes, int inicio) {
    unsigned long long decimal = 0;
    // Convierte la secuencia binaria a su equivalente decimal desplazando y añadiendo cada bit
    for (int i = 0; i < numBits; ++i) {
        decimal = (decimal << 1) | genes[inicio + i];
    }
    double maxPosible = std::pow(2, numBits) - 1;
    // Mapea el entero al rango [min, max]
    return minRango + (decimal / maxPosible) * (maxRango - minRango);
}
```

2.3 Operadores Genéticos

- **Selección:** Se implementó el **Torneo**, eligiendo 2 candidatos al azar y retornando el de menor fitness.
- **Cruza:** Se utiliza un punto de corte aleatorio. Si la probabilidad es menor a 0.7, se intercambian los genes.
- **Mutación:** Se aplica la fórmula del marco teórico.

```cpp
// 5. Mutacion
for (int i = 0; i < h1.genes.size(); ++i) {
    if (prob(rng) < tasaMutacion) {
        h1.genes[i] = std::abs(h1.genes[i] - 1); //  abs(indiv[i]-1)
    }
    if (prob(rng) < tasaMutacion) {
        h2.genes[i] = std::abs(h2.genes[i] - 1);
    }
}
```

2.4 Gestión de Archivos y Automatización

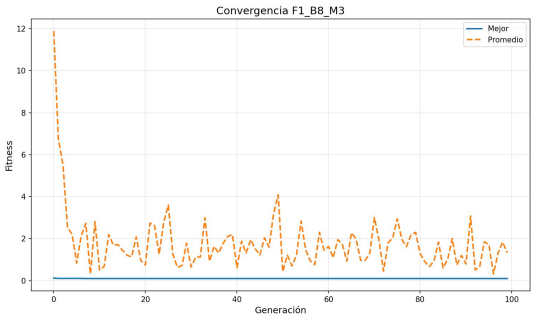
El método `ejecutar()` orquesta la evolución y exporta los datos a archivos CSV (`fit_*.csv` y `pob_*.csv`). Finalmente, el programa invoca un script de **Python (Matplotlib)** mediante `system()` para generar las gráficas automáticamente, facilitando el análisis de los 18 experimentos realizados.

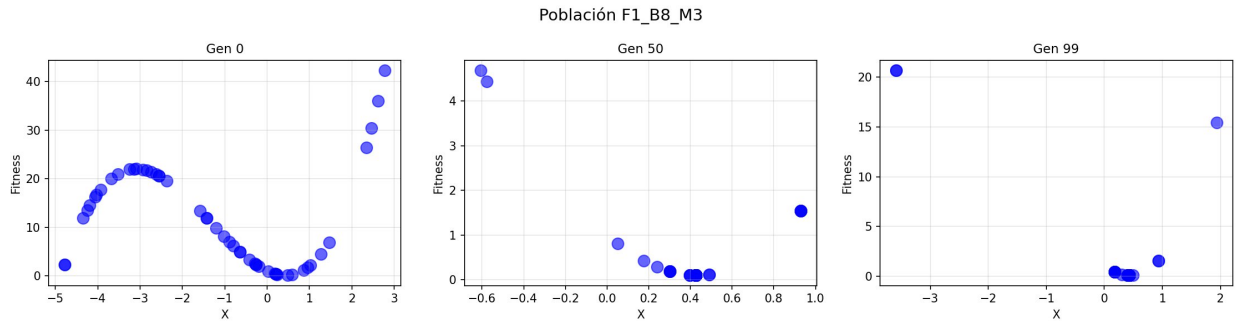
3. Resultados y Gráficas

3.1 $f(x) = x^3+4x^2-4x+1m\ x \in [-5, 3]$

Análisis: Al ser una función polinómica suave, el algoritmo converge rápidamente.

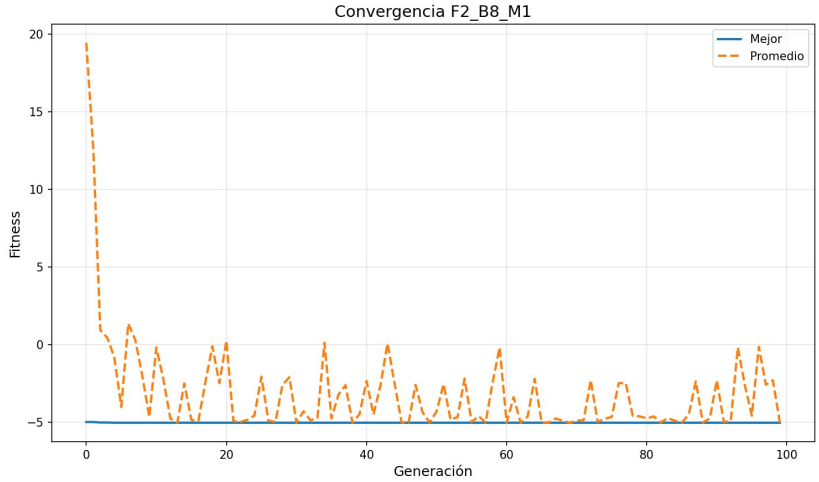
- 4 Bits: Se observa un comportamiento "escalonado" en la gráfica de convergencia debido a la baja resolución.
- 8 Bits: La curva de "Mejor Individuo" es mucho más suave y alcanza un valor más preciso.





3.2 Función 2: $f(x)=x^4+5x^3+4x^2-4x+1$, $x \in [-5, 3]$

Análisis: Similar a la función 1, pero con un rango de búsqueda ligeramente más complejo. La mutación del 1% muestra una convergencia muy rápida, mientras que la del 5% mantiene la diversidad poblacional por más tiempo.

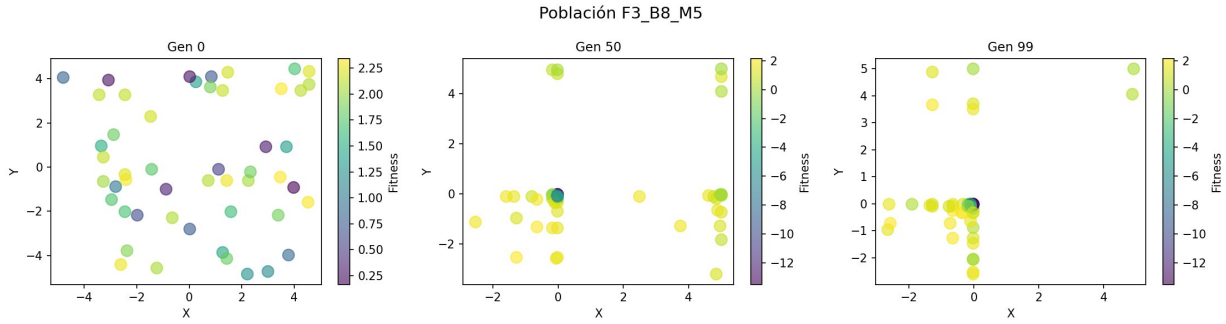


3.3 Función 3: Ackley

Análisis: Esta función presenta múltiples mínimos locales.

- **Dispersión:** En las gráficas de población (scatter plot), se observa claramente cómo los individuos inician dispersos en todo el plano [−5,5] (Gen 0) y progresivamente se agrupan en el centro (0,0) hacia el final.

- **Efecto de la Mutación:** Con 1% de mutación, existe riesgo de estancamiento en mínimos locales. Con 5%, la población sigue explorando zonas cercanas al centro incluso en generaciones avanzadas.



4. Código con comentarios

C++:

```
// 1. Entrada y Salida

#include <iostream>  // Permite imprimir en consola (cout) para ver el progreso de las generaciones y leer entrada (cin).



// 2. Estructuras de Datos

#include <vector>    // Contenedor principal para la población. Permite arreglos dinámicos (resize, push_back) de individuos.



// 3. Matemáticas

#include <cmath>     // Funciones matemáticas para la Función Ackley (exp, sqrt, cos, M_PI).



// 4. Aleatoriedad

#include <random>    // Generación de números aleatorios moderna (Mersenne Twister). Vital para la inicialización, cruza y mutación.



// 5. Algoritmos
```

```cpp
#include <algorithm> // Herramientas para ordenar la población (sort) por fitness o encontrar el mejor individuo (min_element).



// 6. Archivos

#include <fstream>   // Manejo de archivos (File Stream). Se usa para guardar los datos (CSV/TXT) que el script de Python leerá después.



// 7. Cadenas de Texto

#include <string>    // Manipulación de texto. Útil para generar nombres de archivos dinámicos (ej: "gen_01.csv", "gen_02.csv").



// 8. Sistema

#include <cstdlib>   // Interacción con el sistema operativo. Permite usar 'system()' para ejecutar comandos de terminal (como llamar al script de Python).



// Constantes matemáticas necesarias porque no siempre están definidas en todos los compiladores

const double PI = 3.14159265358979323846;

const double E = 2.71828182845904523536;



// ESTRUCTURA DEL INDIVIDUO

// Representa una posible solución al problema.

struct Individuo {

    std::vector<int> genes;      // El "ADN" (Ceros y Unos). Ejemplo: [1, 0, 1, 1]

    std::vector<double> variables; // El valor real traducido. Ejemplo: [2.5]

    double fitness;              // La "calificación". Qué tan buen resultado da en la función.

};



class AlgoritmoGenetico {
```

```
private:
    int idFuncion;          // Cuál función estamos resolviendo (1, 2 o 3)
    int numBits;            // Resolución del genoma (4 u 8 bits)
    double tasaMutacion;    // Probabilidad de cambio genético (1%, 3%, 5%)
    int tamPoblacion;       // Cuántos individuos viven en cada generación
    int maxGeneraciones;    // Cuándo detener la evolución
    int numVariables;       // 1 variable (x) o 2 variables (x, y)
    double minRango, maxRango; // Límites del terreno de búsqueda [-5, 3] o [-5, 5]

    std::vector<Individuo> poblacion; // La comunidad actual de soluciones
    std::mt19937 rng; // Motor de números aleatorios avanzado (Mersenne Twister)

public:
    // CONSTRUCTOR: Configura el experimento inicial
    AlgoritmoGenetico(int func, int bits, double mut, int popSize, int gens)
        : idFuncion(func), numBits(bits), tasaMutacion(mut), tamPoblacion(popSize), maxGenera-
ciones(gens) {

        std::random_device rd;
        rng = std::mt19937(rd()); // Iniciamos la "semilla" aleatoria

        // Ajustamos las reglas del juego según la función elegida
        if (idFuncion == 3) { // Función Ackley (Compleja, 2 variables)
            numVariables = 2;
            minRango = -5.0;
            maxRango = 5.0;
        } else { // Funciones Polinómicas (Simples, 1 variable)
```

```
            numVariables = 1;
            minRango = -5.0;
            maxRango = 3.0;
        }
    }

    // DECODIFICADOR: El traductor de Binario a Decimal
    // Convierte el lenguaje de la máquina (bits) al lenguaje del problema (números reales)
    double decodificar(const std::vector<int>& genes, int inicio) {
        unsigned long long decimal = 0;

        // Paso 1: Convertir binario a entero usando desplazamiento de bits (Bitwise Shift)
        // Es más eficiente que usar potencias matemáticas.
        for (int i = 0; i < numBits; ++i) {
            decimal = (decimal << 1) | genes[inicio + i];
        }

        // Paso 2: Regla de tres para ajustar el entero al rango real [-5, 3]
        double maxPosible = std::pow(2, numBits) - 1;
        return minRango + (decimal / maxPosible) * (maxRango - minRango);
    }

    // FUNCIÓN OBJETIVO:
    // Evalúa qué tan buena es una solución (x, y). Buscamos el valor más bajo (mínimo).
    double evaluar(double x, double y = 0) {
        if (idFuncion == 1) {
            return std::pow(x, 3) + 4 * std::pow(x, 2) - 4 * x + 1;
```

```
    } else if (idFuncion == 2) {
        return std::pow(x, 4) + 5 * std::pow(x, 3) + 4 * std::pow(x, 2) - 4 * x + 1;
    } else {
        // Función de Ackley: Muchos mínimos locales tramposos y un solo mínimo global.
        double t1 = -20 * std::exp(-20 * std::sqrt(0.5 * (x*x + y*y)));
        double t2 = -std::exp(0.5 * (std::cos(2*PI*x) + std::cos(2*PI*y)));
        return E + t1 + t2;
    }
}

// GÉNESIS: Crea la primera generación
void inicializar() {
    poblacion.clear();
    std::uniform_int_distribution<int> dist(0, 1); // Lanzar moneda: 0 o 1

    for (int i = 0; i < tamPoblacion; ++i) {
        Individuo ind;
        // Llenamos su ADN con bits al azar
        for (int b = 0; b < numBits * numVariables; ++b) {
            ind.genes.push_back(dist(rng));
        }
        calcularFitness(ind); // Evaluamos qué tan bueno nació
        poblacion.push_back(ind);
    }
}

// EVALUADOR INDIVIDUAL
```

```
// Proceso completo: Decodificar ADN -> Obtener Variables -> Calcular Fitness
void calcularFitness(Individuo& ind) {
    ind.variables.clear();
    double x = decodificar(ind.genes, 0); // Sacamos X
    ind.variables.push_back(x);

    double y = 0;
    if (numVariables > 1) { // Si es Ackley, sacamos Y también
        y = decodificar(ind.genes, numBits);
        ind.variables.push_back(y);
    }

    ind.fitness = evaluar(x, y);
}

// SELECCIÓN: Torneo
// Enfrentamos a 2 individuos al azar y gana el que tenga MEJOR fitness (menor valor)
Individuo torneo() {
    std::uniform_int_distribution<int> dist(0, tamPoblacion - 1);
    Individuo a = poblacion[dist(rng)];
    Individuo b = poblacion[dist(rng)];
    return (a.fitness < b.fitness) ? a : b;
}

// EVOLUCIÓN: El ciclo de la vida
void evolucionar() {
    std::vector<Individuo> nuevaPoblacion;
```

```
        // 1. ELITISMO
        // Ordenamos y pasamos al mejor individuo directamente para no perderlo.
        std::sort(poblacion.begin(), poblacion.end(), [](const Individuo& a, const Individuo& b)
{
            return a.fitness < b.fitness;
        });
        nuevaPoblacion.push_back(poblacion[0]);

        // Probabilidades y distribuciones para cruza y mutación
        std::uniform_real_distribution<double> prob(0.0, 1.0);
        std::uniform_int_distribution<int> crossDist(1, (numBits * numVariables) - 1);

        // Llenamos el resto de la nueva población con hijos
        while (nuevaPoblacion.size() < tamPoblacion) {
            // A. Selección de Padres
            Individuo p1 = torneo();
            Individuo p2 = torneo();

            // Si los padres son idénticos, intentamos buscar un p2 diferente
            int intentos = 0;
            while (p1.genes == p2.genes && intentos < 10) {
                p2 = torneo(); // Volvemos a sortear al segundo padre
                intentos++;
            }

            Individuo h1 = p1; // Los hijos inician como copias
```

```
        Individuo h2 = p2;

        // B. Cruza (Crossover) - Probabilidad 70%
        if (prob(rng) < 0.7) {
            int punto = crossDist(rng); // Elegimos dónde cortar el ADN
            // Intercambiamos la cola de los padres
            for (int i = punto; i < p1.genes.size(); ++i) {
                h1.genes[i] = p2.genes[i];
                h2.genes[i] = p1.genes[i];
            }
        }

        // C. Mutación - Probabilidad según configuración (1%, 3%, 5%)
        // Recorremos cada bit para ver si "muta" (se invierte)
        for (int i = 0; i < h1.genes.size(); ++i) {
            if (prob(rng) < tasaMutacion) h1.genes[i] = std::abs(h1.genes[i] - 1);
            if (prob(rng) < tasaMutacion) h2.genes[i] = std::abs(h2.genes[i] - 1);
        }

        // D. Nacimiento
        calcularFitness(h1);
        calcularFitness(h2);
        nuevaPoblacion.push_back(h1);

        // Solo insertamos al segundo hijo si aún hay espacio
        if(nuevaPoblacion.size() < tamPoblacion) nuevaPoblacion.push_back(h2);
    }
```

```
        poblacion = nuevaPoblacion; // La nueva generación reemplaza a la vieja
    }

    // FUNCIÓN PRINCIPAL DEL ALGORITMO
    void ejecutar() {
        inicializar();

        // Creamos un nombre de archivo único para guardar los resultados de este experimento
        // Ejemplo: "F1_B4_M1"
        std::string config = "F" + std::to_string(idFuncion) + "_B" + std::to_string(numBits) +
"_M" + std::to_string((int)(tasaMutacion*100));

        std::ofstream fitFile("../results/csv/fit_" + config + ".csv"); // Historial de mejora
        std::ofstream pobFile("../results/csv/pob_" + config + ".csv"); // Mapa de individuos

        fitFile << "Gen,Mejor,Promedio\n";
        pobFile << "Gen,X,Y,Fitness\n";

        // Ciclo de Generaciones
        for (int g = 0; g < maxGeneraciones; ++g) {
            double suma = 0, mejor = 1e9;

            // Solo guardamos la población completa al inicio, mitad y fin para no llenar el
disco
            bool guardarPoblacion = (g == 0 || g == maxGeneraciones/2 || g == maxGeneraciones-1);

            for (auto& ind : poblacion) {
```

```
                suma += ind.fitness;
                if (ind.fitness < mejor) mejor = ind.fitness;

                if (guardarPoblacion) {
                    double yVal = (numVariables > 1) ? ind.variables[1] : 0.0;
                    pobFile << g << "," << ind.variables[0] << "," << yVal << "," << ind.fitness
<< "\n";
                }
            }
            // Guardamos el progreso de la generación
            fitFile << g << "," << mejor << "," << (suma/tamPoblacion) << "\n";

            // Si no es la última vuelta, evolucionamos
            if (g < maxGeneraciones - 1) evolucionar();
        }

        fitFile.close();
        pobFile.close();
        std::cout << "  ✓ Experimento terminado: " << config << "\n";
    }
};

int main() {
    // 1. Preparamos el entorno creando carpetas
    system("mkdir -p ../results/csv ../results/plots");

    std::cout << "\n=== Optimizador Genético Binario ===\n";
```

```
    std::cout << "Iniciando serie de pruebas automatizadas...\n\n";


    // 2. Definimos todos los casos de prueba solicitados
    std::vector<int> funciones = {1, 2, 3};
    std::vector<int> bits = {4, 8};
    std::vector<double> mutaciones = {0.01, 0.03, 0.05};


    // 3. Ejecutamos todos los experimentos en cadena (18 combinaciones)
    for(int f : funciones) {
        for(int b : bits) {


            int poblacion;
            int generaciones;


            if (b == 4) {
            // El universo es diminuto (16 opciones).
            poblacion = 10;
            generaciones = 20;  // Converge casi instantáneamente.
        } else {
            // b == 8
            // El universo es mediano (256 opciones).
            poblacion = 40;
            generaciones = 50;
        }


            for(double m : mutaciones) {
                AlgoritmoGenetico algoritmo(f, b, m, poblacion, generaciones);
```

```
                algoritmo.ejecutar();

            }

        }

    }



    std::cout << "\n✓ Cálculos finalizados.\n";

    std::cout << "Generando visualizaciones con Python...\n\n";



    // 4. Llamamos al script de Python para crear las gráficas

    system("cd ../results/csv && python3 ../../src/graficar.py");



    std::cout << "✓ Proceso completo. Gráficas disponibles en results/plots/\n\n";



    return 0;

}
```

Python (gráficos) :

```
import pandas as pd              # Librería para manejar tablas de datos (como Excel pero en
código)

import matplotlib.pyplot as plt  # Librería estándar para crear gráficas científicas

import glob                      # Herramienta para buscar archivos con patrones (ej: *.csv)

import os                        # Para interactuar con el sistema operativo (crear carpetas)



# 1. PREPARACIÓN DEL ENTORNO

# Creamos una carpeta llamada "plots" para guardar las imágenes.

# exist_ok=True evita que el programa falle si la carpeta ya existe.

os.makedirs("plots", exist_ok=True)
```

```
# 2. BÚSQUEDA DE DATOS
# Buscamos todos los archivos que comiencen con "fit_" y terminen en ".csv".
# Estos archivos contienen el historial de evolución (fitness por generación).
archivos_csv = glob.glob("fit_*.csv")

# 3. CICLO DE PROCESAMIENTO
# Repetimos el proceso para cada experimento encontrado (cada archivo)
for archivo_fit in archivos_csv:
    # Extraemos el nombre de la configuración (ej: de "fit_F1_B4_M1.csv" obtenemos "F1_B4_M1")
    config = archivo_fit.replace("fit_", "").replace(".csv", "")

    # --- GRÁFICA 1: CONVERGENCIA (Evolución del Fitness) ---

    # Leemos el CSV y lo convertimos en un DataFrame (tabla)
    df_fit = pd.read_csv(archivo_fit)

    # Configuramos el lienzo de la figura (tamaño 10x6 pulgadas)
    plt.figure(figsize=(10, 6))

    # Dibujamos dos líneas:
    # 1. La línea sólida azul: El MEJOR individuo (debe bajar siempre)
    plt.plot(df_fit['Gen'], df_fit['Mejor'], label='Mejor Global', linewidth=2)
    # 2. La línea punteada naranja: El PROMEDIO de la población (muestra la diversidad)
    plt.plot(df_fit['Gen'], df_fit['Promedio'], label='Promedio Pob.', linestyle='--',
linewidth=2)
```

```
    # Decoración de la gráfica (Títulos, etiquetas y cuadrícula)
    plt.title(f'Convergencia del Algoritmo: {config}', fontsize=14)
    plt.xlabel('Generación (Tiempo)', fontsize=12)
    plt.ylabel('Valor de Fitness (Costo)', fontsize=12)
    plt.legend(fontsize=10) # Muestra el cuadro con los nombres de las líneas
    plt.grid(True, alpha=0.3) # Cuadrícula suave de fondo
    plt.tight_layout() # Ajusta los márgenes automáticamente

    # Guardamos la imagen en la carpeta 'plots' con buena calidad (dpi=150)
    plt.savefig(f"plots/conv_{config}.png", dpi=150)
    plt.close() # Cerramos la figura para liberar memoria RAM

    # --- GRÁFICA 2: POBLACIÓN (Dispersión de Individuos) ---

    # Construimos el nombre del archivo de población correspondiente (ej: "pob_F1_B4_M1.csv")
    archivo_pob = f"pob_{config}.csv"

    # Solo intentamos graficar si el archivo existe
    if os.path.exists(archivo_pob):
        df_pob = pd.read_csv(archivo_pob)

        # Identificamos qué generaciones se guardaron (usualmente: 0, mitad y final)
        generaciones = sorted(df_pob['Gen'].unique())[:3]

        # Creamos una figura con 3 sub-gráficas lado a lado (1 fila, 3 columnas)
        fig, ax = plt.subplots(1, 3, figsize=(15, 4))
```

```
        # Iteramos sobre las 3 generaciones capturadas
        for i, gen in enumerate(generaciones): # i: índice en la lista (0, 1, 2) y gen: gen-
eración (el número real de generación)
            # Filtramos los datos solo para la generación actual
            datos = df_pob[df_pob['Gen'] == gen]

            # CASO A: FUNCIÓN ACKLEY (F3) -> Es 3D, necesitamos ver X vs Y
            if "F3" in config:
                # Usamos un Scatter Plot (dispersión). El color 'c' indica la altura (fitness).
                scatter = ax[i].scatter(datos['X'], datos['Y'], c=datos['Fitness'],
                                        cmap='viridis', s=100, alpha=0.6)
                ax[i].set_xlabel('Coordenada X', fontsize=10)
                ax[i].set_ylabel('Coordenada Y', fontsize=10)
                ax[i].set_title(f'Gen {gen} (Vista Superior)', fontsize=11)
                # Añadimos barra de color solo en la última gráfica para referencia
                if i == 2: plt.colorbar(scatter, ax=ax[i], label='Fitness')

            # CASO B: FUNCIONES 1D (F1, F2) -> Solo X vs Fitness
            else:
                # Graficamos la posición X contra su valor de función
                ax[i].scatter(datos['X'], datos['Fitness'], color='blue', s=100, alpha=0.6)
                ax[i].set_xlabel('Variable X', fontsize=10)
                ax[i].set_ylabel('Fitness f(x)', fontsize=10)
                ax[i].set_title(f'Gen {gen}', fontsize=11)
                ax[i].grid(True, alpha=0.3)

        # Título general para las 3 sub-gráficas
```

```
        plt.suptitle(f'Evolución de la Población: {config}', fontsize=14)

        plt.tight_layout()

        plt.savefig(f"plots/pob_{config}.png", dpi=150)

        plt.close()



print("✓ Gráficas generadas exitosamente en la carpeta 'plots'")
```

5. Conclusiones

Tras realizar los experimentos variando la cantidad de bits (4 y 8) y los porcentajes de mutación (1%, 3%, 5%), se concluye lo siguiente:

1. **Resolución (Bits):** La codificación de **4 bits** demostró ser insuficiente para obtener el mínimo global exacto. Aunque el algoritmo encuentra la "zona" correcta, el error de precisión es alto (efecto de discretización). La codificación de **8 bits** mejoró drásticamente la precisión del resultado final.
2. **Mutación:**
    1. Un **1%** de mutación favorece una convergencia rápida y estable, ideal para funciones unimodales simples (F1 y F2).
    2. Un **5%** de mutación introduce más "ruido" en la gráfica de convergencia, pero resultó vital para la **Función de Ackley (F3)**, ayudando a los individuos a saltar fuera de los mínimos locales y explorar mejor el espacio de búsqueda.
3. **Convergencia :** El uso de **Elitismo** fue fundamental para asegurar que la curva de "Mejor Individuo" fuera siempre descendente (o plana), garantizando que nunca se perdiera la mejor solución encontrada.

En resumen, el Algoritmo Genético Binario demostró ser una herramienta robusta para optimización, donde el equilibrio entre exploración (mutación) y explotación (selección/elitismo) es clave para el éxito.

5. Referencias

1. **Apuntes de Clase**
2. https://prateek-mishra.medium.com/binary-genetic-algorithm-19936e755271
3. https://es.wikipedia.org/wiki/Algoritmo_gen%C3%A9tico

4. https://es.wikipedia.org/wiki/Selecci%C3%B3n_por_torneos